

Natural Language Processing

Please refer to the video lecture for the full information on this topic and code set-up.

In this lecture we will show an example of how you could use R to perform Natural Language Processing. There is no assignment or project attached to this topic because this topic can range widely. You should also note that a lot of times R is not the best choice for NLP, and that other languages such as Python are a stronger choice due to their library support.

If you are looking for a supplemental assignment, read through the great walkthrough on NLP written [here](#).

Install the necessary libraries

We'll need the following libraries:

- tm
- twitterR
- wordcloud
- RColorBrewer
- e1017
- class

You can install them with this code (uncomment it first):

In [54]:

```
#install.packages('tm',repos='http://cran.us.r-project.org')
#install.packages('twitterR',repos='http://cran.us.r-project.org')
#install.packages('wordcloud',repos='http://cran.us.r-project.org')
#install.packages('RColorBrewer',repos='http://cran.us.r-project.org')
#install.packages('e1017',repos='http://cran.us.r-project.org')
#install.packages('class',repos='http://cran.us.r-project.org')
```

Create a Twitter App

This project requires you to create a twitter account and a twitter application if you want to follow along. Let's outline the steps to do this:

1. Create an Account on Twitter
2. Create a new app at: <https://apps.twitter.com/>
3. You may need to point it to a personal URL, in which case you may need to create a wordpress site or something similar
4. Get Your Keys Under the **Keys and Access Tokens** tab
5. Then use them with the twitterR library:

```
getTwitterOAuth(consumer_key, consumer_secret)
```

Regular Expression Review

Now let's review a few key Regular Expression functions we've touched upon earlier:

grep()

Return the index location of pattern matches

In [9]:

```
args(grep)
```

Out[9]:

```
function (pattern, x, ignore.case = FALSE, perl = FALSE, value = FALSE,  
         fixed = FALSE, useBytes = FALSE, invert = FALSE)  
NULL
```

In [13]:

```
grep('A', c('A', 'B', 'C', 'D', 'A'))
```

Out[13]:

```
1 5
```

nchar()

length of a string

In [14]:

```
args(nchar)
```

Out[14]:

```
function (x, type = "chars", allowNA = FALSE, keepNA = FALSE)  
NULL
```

In [17]:

```
nchar('helloworld')
```

Out[17]:

```
10
```

In [18]:

```
nchar('hello world')
```

Out[18]:

```
11
```

gsub()

perform replacement of the matching patterns

In [19]:

```
args(gsub)
```

Out[19]:

```
function (pattern, replacement, x, ignore.case = FALSE, perl = FALSE,  
         fixed = FALSE, useBytes = FALSE)  
NULL
```

In [20]:

```
gsub('pattern','replacement','hello have you seen the pattern here?')
```

Out[20]:

'hello have you seen the replacement here?'

Text Manipulation

paste()

concatenate several strings together

In [32]:

```
print(paste('A','B','C',sep='...'))
```

[1] "A...B...C"

In [30]:

```
#help(paste)
```

substr()

returns the substring in the given character range start:stop for the given

In [33]:

```
substr('abcdefg',start=2,stop = 5)
```

Out[33]:

'bcde'

strsplit()

splits a string into a list of substrings based on another string split in x

In [49]:

```
strsplit('2016-01-23',split='-')
```

Out[49]:

1. '2016' '01' '23'

NLP Important Terms and Concepts

- Document - The individual text document (e.g. a resume)
- Corpus - The collection of documents (e.g. A group of resumes)
- Bag-of-Words - unordered collection of words (e.g. list of unordered words)
- n-grams - contiguous sequence of n items from a given sequence of text (e.g. ['A','G','C','T'])

- Stopwords - words that appear too often to be of great importance (e.g. the,a,I,etc...)
- Tokens - Any combination of characters (words)
- Stemming - Process to remove suffixes of words (e.g. run,runner,running all reduce to base word - run)
- TF-IDF : term frequency-inver document frequency is a statistic that tells how important a word is in a given corpus, its a way of determining high-information words
- Term Document Matrix - representation of a document collection as vectors

Twitter Mining

Let's mine twitter for some general data!

Step 1: Import Libraries

In []:

```
library(twitteR)
library(tm)
library(wordcloud)
library(RColorBrewer)
```

Step 2: Search for Topic on Twitter

We'll use the twitteR library to data mine twitter. First you need to connect by setting up your Authorization keys and tokens.

In []:

```
setup_twitter_oauth(consumer_key, consumer_secret, access_token=NULL, access_secret=NULL)
```

We will search twitter for the term 'soccer'

In []:

```
soccer.tweets <- searchTwitter("soccer", n=2000, lang="en")
soccer.text <- sapply(soccer.tweets, function(x) x$getText())
```

Step 3: Clean Text Data

We'll remove emoticons and create a corpus

In []:

```
soccer.text <- iconv(soccer.text, 'UTF-8', 'ASCII') # remove emoticons
soccer.corpus <- Corpus(VectorSource(soccer.text)) # create a corpus
```

Step 4: Create a Document Term Matrix

We'll apply some transformations using the TermDocumentMatrix Function

In []:

```
term.doc.matrix <- TermDocumentMatrix(soccer.corpus,
                                       control = list(removePunctuation = TRUE,
                                                         stopwords = c("soccer", "http", stopwords("english")),
                                                         removeNumbers = TRUE, tolower = TRUE))
```

Step 5: Check out Matrix

In []:

```
head(term.doc.matrix)
```

In []:

```
term.doc.matrix <- as.matrix(term.doc.matrix)
```

Step 6: Get Word Counts

In []:

```
word.freqs <- sort(rowSums(term.doc.matrix), decreasing=TRUE)  
dm <- data.frame(word=names(word.freqs), freq=word.freqs)
```

Step 7: Create Word Cloud

In []:

```
wordcloud(dm$word, dm$freq, random.order=FALSE, colors=brewer.pal(8, "Dark2"))
```

Great Job!